

Introduction

Ion is a modern system shell that features a simple, yet powerful, syntax. It is written entirely in Rust, which greatly increases the overall quality and security of the shell, eliminating the possibilities of a [ShellShock](#)-like vulnerability, and making development easier. It also offers a level of performance that exceeds that of Dash, when taking advantage of Ion's features. While it is developed alongside, and primarily for, RedoxOS, it is a fully capable on other *nix platforms, and we are currently searching for a Windows developer to port it to Windows.

Goals

Syntax and feature decisions for Ion are made based upon three measurements:

1. Is the feature useful?
2. Is it simple to use?
3. Will it's implementation be efficient to parse and execute?

A feature is considered useful if there's a valid use case for it, in the concept of a shell language. The syntax for the feature should be simple for a human to read and write, with extra emphasis on readability, given that **most time is spent reading scripts** than writing them. The implementation should require *minimal to zero heap allocations*, and be implemented in a manner that requires *minimal CPU cycles* (so long as it's also **fully documented** and **easy to maintain!**).

It should also be taken into consideration that *shells operate entirely upon strings*, and therefore should be fully equipped for all manner of string manipulation capabilities. That means that users of a shell should not immediately need to grasp for tools like **cut**, **sed**, and **awk**. Ion offers a great deal of control over slicing and manipulating text. Arrays are treated as first class variables with their own unique **@** sigil. Strings are also treated as first class variables with their own unique **\$** sigil. Both support being sliced with **[range]**, and they each have their own supply of methods.

Why Not POSIX?

If Ion had to follow POSIX specifications, it wouldn't be half the shell that it is today, and there'd be no solid reason to use Ion over any other existing shell, given that it'd basically be the same as every other POSIX shell. Redox OS itself doesn't follow POSIX specifications, and neither does it require a POSIX shell for developing Redox's userspace. It's therefore not meant to be used as a drop-in replacement for Dash or Bash. You should retain Dash/Bash on your system for execution of Dash/Bash scripts, but you're free to write new scripts for Ion, or use Ion as the

interactive shell for your user session. Redox OS, for example, also contains Dash for compatibility with software that depends on POSIX scripts.

That said, Ion's foundations are heavily inspired by POSIX shell syntax. If you have experience with POSIX shells, then you already have a good idea of how most of Ion's core features operate. A quick sprint through this documentation will bring you up to speed on the differences between our shell and POSIX shells. Namely, we carry a lot of the same operators: `$`, `|`, `||`, `&`, `&&`, `>`, `<`, `<<`, `<<<`, `$()`, `$((())`. Yet we also offer some functionality of our own, such as `@`, `@()`, `$method()`, `@method()`, `^|`, `^>`, `&>`, `&|`. Essentially, we have taken the best components of the POSIX shell specifications, removed the bad parts, and implemented even better features on top of the best parts. That's how open source software evolves: iterate, deploy, study, repeat.

Migrating from POSIX Shells

Notable changes

- Arrays are full-class citizens, using the @ sigil. That means emails and git urls must be single quoted
- The shell has proper scopes (variables get unset after the end of the definition scope), and functions are closures
- The shell has an internal variable store. That means environment variables must be explicitly exported to be available to commands.
- For now, per-command environment variables are not supported (ex: `LANG=it_CH.utf8`
`man man`)
- The testing builtin (`[[ .. ]]`) was replaced with `test`, `exists`, and/or other commands
- The control flow have been revisited, see the relevant part of the manual

Customizing your prompt

- Define the PROMPT function to be called whenever the prompt needs to be drawn. Simply print the prompt to stdout in the function (printf or git branch directly)
- Variables are defined with all the colors (see the namespaces manual page for all details). This means you don't have to deal with all the escape codes directly. No more `\x033[33;m`, instead it's `${color::yellow}`.

General rules

Performance: Let Arithmetic vs Arithmetic Expansions

let arithmetic is generally faster than **\$(())** expansions. The arithmetic expansions should be used for increasing readability, or more complex arithmetic. If speed is important: Multiple *let arithmetic statements will tend to be faster* than a single arithmetic expansion.

Quoting Rules

- Variables are expanded in double quotes, but not single quotes.
- Braces are expanded when unquoted, but not when quoted.

XDG App Dirs Support

All files created by Ion can be found in their respective XDG application directories. For example, the init file for Ion can be found in **\$HOME/.config/ion/initrc** on Linux systems; and the history file can be found at **\$HOME/.local/share/ion/history**. On the first launch of Ion, a message will be given to indicate the location of these files.

Read-eval-print loop

Implicit `cd`

Like the [Friendly Interactive Shell](#), Ion also supports executing the `cd` command automatically when given a path. Paths are denoted by beginning with `./`/`../``~/`, or ending with `/`.

```
~/Documents # cd ~/Documents
..          # cd ..
.config     # cd .config
examples/   # cd examples/
```

Multi-line Arguments

If a line in your script becomes too long, appending `\` will make Ion ignore newlines and continue reading the next line.

```
command arg arg2 \
  arg3 arg4 \
  arg 5
```

Multi-line Strings

If a string needs to contain newlines, you use an open quote. Ion will only begin parsing supplied commands that are terminated. Either double or single quotes can be used.

```
echo "This is the first line
     this is the second line
     this is the third line"
```

Prompt Function

The prompt may optionally be generated from a function, instead of a string. Due to the need to perform a fork and capture of its output as prompt, prompts generated from functions aren't

as efficient. Below the requirement to use the function with name **PROMPT**:

```
fn PROMPT
  echo -n "${PWD}# "
end
```

Key Bindings

There are two pre-set key maps available: **Emacs (default)** and **Vi**. You can switch between them with the `keybindings` built-in command.

```
keybindings vi
keybindings emacs
```

Vi keybinding: You can define the displayed indicator for normal and insert modes with the following variables:

```
$ export VI_NORMAL = "[=] "
$ export VI_INSERT = "[+] "
$ keybindings vi
[+] $
```

Variables

The `let` builtin is used to create local variables within the shell, and apply basic arithmetic to variables. The `export` keyword may be used to do the same for the creation of external variables. Variables cannot be created the POSIX way, as the POSIX way is awkward to read/write and parse.

```
let string_variable = "hello string"
let array_variable = [ hello array ]
echo $string_variable
echo @array_variable
```

```
hello string
hello array
```

Multiple Assignments

Ion also supports setting multiple values at the same time

```
let a b = one two
echo $a
echo $b

let a b = one [two three four]
echo $a
echo @b
```

```
one
two
one
two three four
```

Type-Checked Assignments

It's also possible to designate the type that a variable is allowed to be initialized with. Boolean type assignments will also normalize inputs into either `true` or `false`. When an invalid value is supplied, the assignment operation will fail and an error message will be printed. All assignments after the failed assignment will be ignored.

```
let a:bool = 1
let b:bool = true
let c:bool = n
echo $a $b $c
let fail:bool = ""

let a:str b:[str] c:int d:[float] = one [two three] 4 [5.1 6.2 7.3]
echo $a
echo @b
echo $c
echo @d
```

```
true true false
ion: assignment error: fail: expected bool
one
two three
4
5.1 6.2 7.3
```

Dropping Variables

Variables may be dropped from a scope with the `drop` keyword. Considering that a variable can only be assigned to one type at a time, this will drop whichever value is assigned to that type.

```
let string = "hello"
drop string
let array = [ hello world ]
drop array
```

Supported Primitive Types

- `str`: A string, the essential primitive of a shell.
- `bool`: A value which is either `true` or `false`.
- `int`: An integer is any whole number.
- `float`: A float is a rational number (fractions represented as a decimal).

Arrays

The `[T]` type, where `T` is a primitive, is an array of that primitive type.

Maps

Likewise, `hmap[T]` and `bmap[T]` work in a similar fashion, but are a collection of key/value pairs, where the key is always a `str`, and the value is defined by the `T`.

String Variables

We can evaluate expressions to assign their result to the variable and print with with `$` sigil. Read the chapter expansions for more information about the expansion behavior.

```
# The CI can not handle deletions properly.  
mkdir -p _tmp _tmp/t1 _tmp/t2  
cd _tmp  
let filelist = *  
echo $filelist  
cd ..
```

```
t1 t2
```

Slicing a string.

Strings can be sliced in Ion using a range.

```
let foo = "Hello, World"  
echo $foo[..5]  
echo $foo[7..]  
echo $foo[2..9]
```

```
Hello  
World  
llo, Wo
```

String concatenation

The `++=` and `::=` operators can be used to efficiently concatenate a string in-place.

```
let string = world  
echo $string  
let string += !  
echo $string  
let string ::= "Hello, "  
echo $string
```

```
world  
world!  
Hello, world!
```

Array Variables

The `[]` syntax in Ion denotes that the contents within should be parsed as an array expression. On using `let` keyword for array variables, all array arguments must be wrapped within the `[]` syntax. Otherwise it will be coerced into a space-separated string. This design decision was made due to the possibility of an expanded array with one element being interpreted as a string.

Once created, you may call an array variable in the same manner as a string variable, but you must use the `@` sigil instead of `$`. When expanded, arrays will be expanded into multiple arguments. Hence it is possible to use arrays to set multiple arguments in commands.

NOTE If an array is double quoted, it will be coerced into a string. This behavior is equivalent to invoking the `$join(array)` method.

NOTE: Brace expansions also create arrays.

Create a new array

Arguments enclosed within brackets are treated as elements within an array.

```
let array = [ one two 'three four' ]
echo @array
```

```
one two three four
```

Indexing into an array

Values can be fetched from an array via their position in the array as the index.

```
let array = [ 1 2 3 4 5 ]
echo @array[0]
echo @array[2..=4]
```

```
1
3 4 5
```


Copy array into a new array

Passing an array within brackets enables performing a deep copy of that array.

```
let array = [ 1 2 3 ]
let array_copy = [ @array ]
echo @array_copy
```

```
1 2 3
```

Array join

This will join each element of the array into a string, adding spaces between each element.

```
let array = [ hello world ]
let other_array = [ this is the ion ]
let array = [ @array @other_array shell ]
let as_string = @array
echo @array
echo $as_string
```

```
hello world this is the ion shell
hello world this is the ion shell
```

Array concatenation and variable stripping

The `++=` and `::=` operators can be used to efficiently concatenate an array in-place.

```
let array = [2 3]
let array += [4 5] # append
let array ::= [0 1] # append before beginning [0 1]
let array \|= [2 3] # remove variables 2 and 3
echo @array
let array += 6 # same with single variables
let array ::= -1
let array \|= 0
echo @array
```

```
0 1 4 5
-1 1 4 5 6
```

Practical array usage

Passing arrays as command arguments and capturing output of commands as arrays is useful.

```
mkdir -p _tmp _tmp/t1 _tmp/t2
cd _tmp
let args = [-a --file-type]
ls @args      # use args as arguments for command ls
let res = [ @(ls) ] # get result of ls as array res
echo @res     # output the array res
cd ..
rm -fr _tmp
```

```
./
../
t1/
t2/
t1 t2
```

Maps

Maps, (AKA dictionaries), provide key-value data association. Ion has two variants of maps: Hash and BTree. Hash maps are fast but store data in a random order. BTree maps are slower, but keep their data in a sorted order. If not sure what to use, go with Hash maps.

Creating maps uses the same right-hand-side array syntax. However for design simplicity, users must annotate the type to translate the array into a map.

Please note, the map's inner type specifies the value's type and not of the key. Keys will always be typed `str`.

HashMap

```
let hashmap:hmap[str] = [ blue=pc27 red=pc2 green=pc15 ]
let x = blue
echo @hashmap[$x] @hashmap[red] # fetch values
let hashmap[orange] = pc22 # add new key with value
#echo @keys(hashmap) #get keys
#echo @values(hashmap) #get values
#echo @hashmap #get keys and values
#for key value in @hashmap #use keys and values
#  echo $key: $value
#end
```

```
pc27 pc2
```

BTreeMap

```
let btreemap:bmap[str] = [ pc2=red pc15=green pc27=blue ]
let x = pc2
echo @btreemap[$x] @btreemap[pc15] # fetch values
let btreemap[orange] = pc22 # add new key with value
echo @keys(btreemap) #get keys
echo @values(btreemap) #get values
echo @btreemap #get keys and values
for key value in @btreemap #use keys and values
  echo $key: $value
end
```

```
red green
orange pc15 pc2 pc27
pc22 green red blue
orange pc22 pc15 green pc2 red pc27 blue
orange: pc22
pc15: green
pc2: red
pc27: blue
```

Let Arithmetic

Ion supports applying some basic arithmetic, one operation at a time, to string variables. To specify to `let` to perform some arithmetic, designate the operation immediately before `=`. Operators currently supported are:

- ☒ Add (+)
- ☒ Subtract (-)
- ☒ Multiply (*)
- ☒ Divide (/)
- ☒ Integer Divide (//)
- ☐ Modulus (%)
- ☒ Powers (**)

Individual Assignments

The following examples are a demonstration of applying a mathematical operation to an individual variable.

```
let value = 5
echo $value
let value += 5
echo $value
let value -= 2
echo $value
let value *= 2
echo $value
let value /= 2
echo $value
let value **= 2
echo $value
let value /= 2
echo $value
```

```
5
10
8
16
8
64.0
32.0
```

Multiple Assignments

It's also possible to perform a mathematical operation to multiple variables. Each variable will be designated with a paired value.

```
let a b = 5 5
echo $a $b
let a b += 5 5
echo $a $b
let a b -= 2 2
echo $a $b
let a b *= 2 2
echo $a $b
let a b /= 2 2
echo $a $b
let a b **= 2 2
echo $a $b
let a b /= 2 2
echo $a $b
```

```
5 5
10 10
8 8
16 16
8 8
64.0 64.0
32.0 32.0
```

Exporting Variables

The `export` builtin operates identical to the `let` builtin, but it does not support arrays, and variables are exported to the OS environment.

```
export GLOBAL_VAL = "this"
```

Scopes

A scope is a batch of commands, often ended by `end`. Things like `if`, `while`, etc all take a scope to execute.

In ion, just like most other languages, all variables are destroyed once the scope they were defined in is gone. Similarly, variables from other scopes can still be overridden. However, ion has no dedicated keyword for updating an existing variable currently, so the first invocation of `let` gets to "own" the variable.

This is an early implementation and will be improved upon with time

```
let x = 5 # defines x

# This will always execute.
# Only reason for this check is to show how
# variables defined inside it are destroyed.
if test 1 == 1
  let x = 2 # updates existing x
  let y = 3 # defines y

  # end of scope, y is deleted since it's owned by it
end

echo $x # prints 2
echo $y # prints nothing, y is deleted already
```

Functions

Functions have the scope they were defined in. This ensures they don't use any unintended local variables that only work in some cases. Once again, this matches the behavior of most other languages, apart from perhaps LOLCODE.


```
let x = 5 # defines x

fn print_vars
  echo $x # prints 2 because it was updated before the function was called
  echo $y # prints nothing, y is owned by another scope
end

if test 1 == 1
  let x = 2 # updates existing x
  let y = 3 # defines y
  print_vars
end
```

Namespaces (colors, scopes and environment variables)

Various functionalities are exposed via namespaces. They are currently colors, scopes and environment variables.

Syntax

To access namespaces, simply use `${namespace::variable}`.

Colors (c/color namespace)

Ion aims to make it easy to make your own prompts, without having to use an external script because of its length. One of the features that make this goal possible is a simple but powerful integration of colors via variables.

Colors available are:

- black
- blue
- cyan
- dark_gray
- default
- green
- magenta
- red
- yellow
- light_blue
- light_cyan
- light_gray
- light_green
- light_magenta
- light_red
- light_yellow

To change the background color, simply append bg to the color (ex: `${c::black}` => `${c::blackbg}`)

Attributes for the command line are also available:

- blink
- bold
- dim
- hidden
- reverse
- underlined

You can also access the full 256 colors using hex or decimal representation. For example `${c::F}` accesses the 16th color, `${c::0xFF}` the 255th, and `${c::14}` would use color #14.

Lastly, you can use true colors using hexes. `${c::0x000000}` and `${c::0x000}` would print pure black independent of the terminal's color scheme. It should be advised to avoid using those colors except specific use cases where the exact color is required.

As a last tip, you can delimit different attributes using commas, so `${c::black}${c::redbg}` is also `${c::black,redbg}`.

Example

```
fn PROMPT
  printf "${c::red}${c::bluebg}${c::bold}%s${c::reset}" $(git branch)
end
```

would print the git branches in bold red over a blue background.

Scopes (super and global namespaces)

Since Ion has proper scoping contrary to other shells, helpers are provided to access variables in various scopes. The super namespaces crosses a function boundary and the global namespace accesses the global scope. This allows variable shadowing.

Example

```
let a = 1

fn demo
  let a = 2

  fn nested
    let a = 3
    echo ${global::a}
    echo ${super::a}
    echo $a
  end
  nested
end

demo
```

```
1
2
3
```

Environment variable (env namespace)

Ion errors when users access undefined variables. Usually, though, environment variables can't be predicted. It is also clearer to define where they are used. As such, the env namespace will simply emit an empty string if the environment variable is not defined.

Example

```
echo ${env::SHELL}
```

would output `/usr/local/bin/ion` on a system with a locally built Ion as login shell.

Expansions

Expansions provide dynamic string generation capabilities. These work identical to the standard POSIX way, but there are a few major differences: arrays are denoted with an @ sigil, and have their own variant of process expansions (@()) which splits outputs by whitespace; the arithmetic logic is more feature-complete, supports floating-point math, and handles larger numbers; and Ion supports methods in the same manner as the [Oil shell](#).

Variable Expansions

Expansions provide dynamic string generation capabilities. These work identical to the standard POSIX way, but there are a few major differences: arrays are denoted with an `@` sigil, and have their own variant of process expansions (`@()`) which splits outputs by whitespace; and our arithmetic logic is destined to be both more feature-complete, supports floating-point math, and handles larger numbers.

String Variables

Like POSIX shells, the `$` sigil denotes that the following expression will be a string expansion. If the character that follows is an accepted Unicode character, all characters that follow will be collected until either a non-accepted Unicode character is found, or all characters have been read. Then the characters that were collected will be used as the name of the string variable to substitute with.

```
let string = "example string"
echo $string
echo $string:$string
```

```
example string
example string:example string
```

NOTE:

- Accepted characters are **unicode** alphanumeric characters and `_`.

Array Variables

Unlike POSIX, Ion also offers support for first class arrays, which are denoted with the `@` sigil. The rules for these are identical, but instead of returning a single string, it will return an array of strings. This means that it's possible to use an array variable as arguments in a command, as each element in the array will be treated as a separate shell word.

```
let array = [one two three]
echo @array
```

```
one two three
```

However, do note that double-quoted arrays are coerced into strings, with spaces separating each element. It is equivalent to using the `$join(array)` method. Containing multiple arrays within double quotes is therefore equivalent to folding the elements into a single string.

Braced Variables

Braces can also be used when you need to integrate a variable expansion along accepted Unicode characters.

```
let hello = "hello123"  
echo ${hello}world  
let hello = [hello 123 ' ']  
echo @{{hello}}world
```

```
hello123world  
hello 123  world
```

Aliases

Ion also supports aliasing commands, which can be defined using the `alias` builtin. Aliases are often used as shortcuts to repetitive command invocations.

```
alias ls = "ls --color"  
#echo $ls #ion: expansion error: Variable does not exist  
#alias are stored separately
```

Process Expansions

Ion supports two forms of process expansions: string-based process expansions (`$()`) that are commonly found in POSIX shells, and array-based process expansions (`@()`), a concept borrowed from the Oil shell. Where a string-based process expansion will execute a command and return a string of that command's standard output, an array-based process expansion will split the output into an array delimited by whitespaces.

```
let string = $(cmd args...)
let array = [ @(cmd args...) ]
```

NOTES:

- To split outputs by line, see [@lines\\$\(cmd\)](#).
- `@(cmd)` is equivalent to [@split\\$\(cmd\)](#).

```
mkdir -p _tmp _tmp/t1 _tmp/t2
cd _tmp
let res = $(ls)
let res2 = [ @(ls) ]
echo $res      # output the string
echo @res2     # output the array
cd ..
rm -fr _tmp
```

```
t1
t2
t1 t2
```


Brace Expansions

Sometimes you may want to generate permutations of strings, which is typically used to shorten the amount of characters you need to type when specifying multiple arguments. This can be achieved through the use of braces, where braced tokens are comma-delimited and used as infixes. Any non-whitespace characters connected to brace expansions will also be included within the brace permutations.

NOTE: Brace expansions will not work within double quotes.

```
echo filename.{ext1,ext2}
```

```
filename.ext1 filename.ext2
```

Multiple brace tokens may occur within a braced collection, where each token expands the possible permutation variants.

```
echo job_{01,02}.{ext1,ext2}
```

```
job_01.ext1 job_01.ext2 job_02.ext1 job_02.ext2
```

Brace tokens may even contain brace tokens of their own, as each brace element will also be expanded.

```
echo job_{01_{out,err},02_{out,err}}.txt
```

```
job_01_out.txt job_01_err.txt job_02_out.txt job_02_err.txt
```

Braces elements may also be designated as ranges, which may be either inclusive or exclusive, descending or ascending, numbers or latin alphabet characters.

```
echo {1..10}  
echo {10..1}  
echo {1...10}  
echo {10...1}  
echo {a..d}  
echo {d..a}  
echo {a...d}  
echo {d...a}
```

```
1 2 3 4 5 6 7 8 9
10 9 8 7 6 5 4 3 2
1 2 3 4 5 6 7 8 9 10
10 9 8 7 6 5 4 3 2 1
a b c
d c b
a b c d
d c b a
```

It's also important to note that, as range brace expansions return arrays, they may be used in for loops.

```
for num in {1..10}
  echo $num
end
```

```
1
2
3
4
5
6
7
8
9
```

Arithmetic Expansions

We've exported our arithmetic logic into a separate crate [calculate](#). We use this library for both our `calc` builtin, and for parsing arithmetic expansions. Use `math` if you want a REPL for arithmetic, else use arithmetic expansions (`$(a + b)`) if you want the result inlined. Variables may be passed into arithmetic expansions without the `$` sigil, as it is automatically inferred that text references string variables. Supported operators are as below:

- Add (`$(a + b)`)
- Subtract (`$(a - b)`)
- Divide (`$(a / b)`)
- Multiply (`$(a * b)`)
- Powers (`$(a ** b)`)
- Square (`$(a2)`)
- Cube (`$(a3)`)
- Modulus (`$(a % b)`)
- Bitwise XOR (`$(a ^ b)`)
- Bitwise AND (`$(a & b)`)
- Bitwise OR (`$(a | b)`)
- Bitwise NOT (`$(a ~ b)`)
- Left Shift (`$(a << b)`)
- Right Shift (`$(a >> b)`)
- Parenthesis (`$(4 * (pi * r2))`)

Take note, however, that these expressions are evaluated to adhere to order of operation rules. Therefore, expressions are not guaranteed to evaluate left to right, and parenthesis should be used when you are unsure about the order of applied operations.

Method Expansions

There are two forms of methods within Ion: [string methods](#) and [array methods](#). Array methods are methods which return arrays, and string methods are methods which return strings. The distinction is made between the two by the sigil that is invoked when calling a method. For example, if the method is denoted by the `$` sigil, then it is a string method. Otherwise, if it is denoted by the `@` sigil, then it is an array method. Example as follows:

```
echo $method_name(variable)
for elem in @method_name(variable); echo $elem; end
```

Methods are executed at the same time as other expansions, so this leads to a performance optimization when combining methods with other methods or expansions. Ion includes a number of these methods for common use cases, but it is possible to create and/or install new methods to enhance the functionality of Ion. Just ensure that systems executing your Ion scripts that require those plugins are equipped with and have those plugins enabled. If you have ideas for useful methods that would be worth including in Ion by default, feel free to open a feature request to discuss the idea.

Methods Support Inline Expressions

So we heard that you like methods, so we put methods in your methods. Ion methods accept taking expressions as their arguments -- both for the input parameter, and any supplied arguments to control the behavior of the method.

```
echo $method($(cmd...) arg)

let string_var = "value in variable"
echo $method(string_var)

echo $method("actual value" arg)
```

Overloaded Methods

Some methods may also perform different actions when supplied a different type. The `$len()` method, for example, will report the number of graphemes within a string, or the number of elements within an array. Ion is able to determine which of the two were provided based on

the first character in the expression. Quoted expressions, and expressions with start with `$`, are strings; whereas expressions that start with either `[` or `@` are treated as arrays.

```
echo $len("a string")
echo $len([1 2 3 4 5])
```

Method Arguments

Some methods may have their behavior tweaked by supplying some additional arguments. The `@split()` method, for example, may be optionally supplied a pattern for splitting.

```
for elem in @split("some space-delimited values"); echo $elem; end
for elem in @split("some, comma-separated, values" ", "); echo $elem; end
```

String Methods

The following are the currently-supported string methods:

- `basename`
- `extension`
- `filename`
- `join`
- `find`
- `len`
- `len_bytes`
- `parent`
- `repeat`
- `replace`
- `replacen`
- `regex_replace`
- `reverse`
- `to_lowercase`
- `to_uppercase`
- `escape`
- `unescape`
- `or`

basename

Defaults to string variables. When given a path-like string as input, this will return the basename (complete filename, extension included). IE: `/parent/filename.ext` ->

```
filename.ext
```

```
echo $(basename("/parent/filename.ext"))
```

```
filename.ext
```

extension

Defaults to string variables. When given a path-like string as input, this will return the extension of the complete filename. IE: `/parent/filename.ext` -> `ext`.

```
echo $extension("/parent/filename.ext")
```

```
ext
```

filename

Defaults to string variables. When given a path-like string as input, this will return the file name portion of the complete filename. IE: `/parent/filename.ext` -> `filename`.

```
echo $filename("/parent/filename.ext")
```

```
filename
```

join

Defaults to array variables. When given an array as input, the join string method will concatenate each element in the array and return a string. If no argument is given, then those elements will be joined by a single space. Otherwise, each element will be joined with a given pattern.

```
let array = [1 2 3 4 5]
echo $join(array)
echo $join(array ", ")
```

```
1 2 3 4 5
1, 2, 3, 4, 5
```

find

Defaults to string variables. When given an string, it returns the first index in which that string appears. It returns `-1` if it isn't contained.

```
echo $find("FOOBAR" "OB")
echo $find("FOOBAR" "ob")
```

```
2
-1
```

len

Defaults to string variables. Counts the number of graphemes in the output. If an array expression is supplied, it will print the number of elements in the array.

```
echo $len("foobar")  
echo $len("❤️")  
echo $len([one two three four])
```

```
6  
1  
4
```

len_bytes

Defaults to string variables. Similar to the `len` method, but counts the number of actual bytes in the output, not the number of graphemes.

```
echo $len_bytes("foobar")  
echo $len_bytes("❤️")
```

```
6  
6
```

parent

Defaults to string variables. When given a path-like string as input, this will return the parent directory's name. IE: `/root/parent/filename.ext` -> `/root/parent`

```
echo $parent("/root/parent/filename.ext")
```

```
/root/parent
```

repeat

Defaults to string variables. When supplied with a number, it will repeat the input N amount of times, where N is the supplied number.

```
echo $repeat("abc, " 3)
```



```
abc, abc, abc,
```

replace

Defaults to string variables. Given a pattern to match, and a replacement to replace each match with, a new string will be returned with all matches replaced.

```
let input = "one two one two"
echo $replace(input one 1)
echo $replace($replace(input one 1) two 2)
```

```
1 two 1 two
1 2 1 2
```

replacen

Defaults to string variables. Equivalent to `replace`, but will only replace the first N amount of matches.

```
let input = "one two one two"
echo $replacen(input "one" "three" 1)
echo $replacen(input "two" "three" 2)
```

```
three two one two
one three one three
```

regex_replace

Defaults to string variables. Equivalent to `replace`, but the first argument will be treated as a regex.

PS: By default, unicode support will be disabled to trim the size of Ion. Add the "unicode" flag to enable it.

```
echo $regex_replace("bob" "^b" "B")
echo $regex_replace("bob" 'b$' "B")
```

```
Bob
boB
```

reverse

Defaults to string variables. Simply returns the same string, but with each grapheme displayed in reverse order.

```
echo $reverse("foobar")
```

```
rabooF
```

to_lowercase

Defaults to string variables. All given strings have their characters converted to an lowercase equivalent, if an lowercase equivalent exists.

```
echo $to_lowercase("FOOBAR")
```

```
foobar
```

to_uppercase

Defaults to string variables. All given strings have their characters converted to an uppercase equivalent, if an uppercase equivalent exists.

```
echo $to_uppercase("foobar")
```

```
FOOBAR
```

escape

Defaults to string variables. Escapes the content of the string.

```
let line = " Mary   had\ta little  \n\t lamb\t"  
echo $escape($line)
```

```
Mary   had\\ta little  \\n\\t lamb\\t
```

unescape

Defaults to string variables. Unescapes the content of the string.

```
let line = " Mary   had\t a little  \n\t lamb\t"  
echo $unescape($line)
```

```
Mary   had       a little  
      lamb
```

or

Defaults to string variables. Fallback to a given value if the variable is not defined or is an empty string.

```
echo $or($unknown_variable "Fallback")  
let var = 42  
echo $or($var "Not displayed")
```

```
Fallback  
42
```

Array Methods

The following are the currently-supported array methods.

- [lines](#)
- [split](#)
- [split_at](#)
- [bytes](#)
- [chars](#)
- [graphemes](#)
- [reverse](#)
- [subst](#)

lines

Defaults to string variables. The supplied string will be split into one string per line in the input argument. This is equivalent to `@split(value '\n')`.

```
echo @lines($unescape("firstline\nsecondline"))
for line in @lines($unescape("third\nfourth\nfifth"))
  echo $line
end
```

```
firstline secondline
third
fourth
fifth
```

split

The supplied string will be split according to a pattern specified as an argument in the method. If no pattern is supplied, then the input will be split by whitespace characters. Useful for splitting simple tabular data.

```
echo @split("onetwoone" "two")
for data in @split("person, age, some data" ", ")
    echo $data
end
for data in @split("person age data")
    echo $data
end
```

```
one one
person
age
some data
person
age
data
```

split_at

Defaults to string variables. The supplied string will be split in two pieces, from the index specified in the second argument.

```
echo @split_at("onetwoone" "3")
echo @split_at("FOOBAR" "3")
#echo @split_at("FOOBAR") #ion: expansion error: split_at: requires an argument
#echo @split_at("FOOBAR" "-1") #ion: expansion error: split_at: requires a valid
number as an argument
#echo @split_at("FOOBAR" "8") #ion: expansion error: split_at: value is out of
bounds
```

```
one twoone
FOO BAR
```

bytes

Defaults to string variables. Returns an array where the given input string is split by bytes and each byte is displayed as their actual 8-bit number.

```
echo @bytes("onetwo")
echo @bytes("abc")
```

```
111 110 101 116 119 111
97 98 99
```

chars

Defaults to string variables. Returns an array where the given input string is split by chars.

```
echo @chars("onetwo")
for char in @chars("foobar")
  echo $char
end
```

```
o n e t w o
f
o
o
b
a
r
```

graphemes

Defaults to string variables. Returns an array where the given input string is split by graphemes.

```
echo @graphemes("onetwo" "3")
for grapheme in @graphemes("foobar")
  echo $grapheme
end
```

```
o n e t w o
f
o
o
b
a
r
```

reverse

Defaults to array variables. Returns a reversed copy of the input array.

```
echo @reverse([1 2 3])
echo @reverse(["a"])
let foo = [1 2 3]
echo @reverse(@foo)
```

```
3 2 1
a
3 2 1
```

subst

Returns the 1. argument if the 1. argument as an array has at least on element.

Returns the 2. argument as the default array if the 1. argument is an empty array.

This methods raises an error

- if no 2 arguments is provided
- if the 1. argument is not an array
- if the 2. argument is not an array

Note: If you want to use this method with an string, use [\\$or](#) method instead or split the string method via [@split](#) before using that method.

```
let empty = []
for number in @subst(@empty [1 2 3])
  echo $number
end
```

```
1
2
3
```

On Redox Os

File scheme

you can also specify absolute paths via the file scheme.

The file scheme has the prefix "file:". A path starting with the prefix "file:" is same as giving an absolute path which starts with "/" on a Linux system for example.

The following command on Redox Os

```
ls file:home/user/*.txt
```

is same as

```
ls /home/user/*.txt
```

Leading "/" will be ignored after the prefix "file:". A path like "file:/something" is the same as "file:something" for ion.

Ranges & Slicing Syntax

Ion supports a universal syntax for slicing strings and arrays. For maximum language support, strings are sliced and indexed by graphemes. Arrays are sliced and indexed by their elements. Slicing uses the same `[]` characters as arrays, but the shell can differentiate between a slice and an array based on the placement of the characters (immediately after an expansion).

NOTE: It's important to note that indexes count from 0, as in most other system programming languages.

NOTE: `...` and `..=` can be used interchangeable.

Exclusive Range

The exclusive syntax will grab all values starting from the first index, and ending on the Nth element, where N is the last index value. The Nth element's ID is always one less than the Nth value.

```
let array = [{1...10}]
echo @array[0..5]
echo @array[..5]

let string = "hello world"
echo $string[..5]
echo $string[6..]
```

```
1 2 3 4 5
1 2 3 4 5
hello
world
```

Inclusive Range

When using inclusive ranges, the end index does not refer to the Nth value, but the actual index ID.

```
let array = [{1...10}]
echo @array[0...5]
```

```
1 2 3 4 5 6
```

The `=` character may be used instead of the third dot.

```
echo @array[0..=5]
```

```
1 2 3 4 5 6
```

Descending Ranges

Ranges do not have to always be specified in ascending order. Descending ranges are also supported. However, at this time you cannot provide an descending range as an index to an array.

```
echo {10...1}  
echo {10..1}
```

```
10 9 8 7 6 5 4 3 2 1  
10 9 8 7 6 5 4 3 2
```

Negative Values Supported

Although this will not work for arrays, you may supply negative values with ranges to create negative values in a range of numbers.

```
echo {-10...10}
```

```
-10 -9 -8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7 8 9 10
```

Stepping Ranges

Stepped ranges are also supported.

Stepping Forward w/ Brace Ranges

Brace ranges support a syntax similar to Bash, where the starting index is supplied, followed by two periods and a stepping value, followed by either another two periods or three periods, then the end index.

```
echo {0..3...12}  
echo {0..3..12}
```

```
0 3 6 9 12  
0 3 6 9
```

Stepping Forward w/ Array Slicing

Array slicing, on the other hand, uses a more Haskell-ish syntax, whereby instead of specifying the stepping with two periods, it is specified with a comma.

```
let array = [{0...30}]  
echo @array[0,3..]
```

```
0 3 6 9 12 15 18 21 24 27 30
```

Stepping In Reverse w/ Brace Ranges

Brace ranges may also specify a range that descends in value, rather than increases.

```
echo {10..-2...-10}  
echo {10..-2..-10}
```

```
10 8 6 4 2 0 -2 -4 -6 -8 -10  
10 8 6 4 2 0 -2 -4 -6 -8
```

Stepping In Reverse w/ Array Slicing

Arrays may also be sliced in reverse order using the same syntax as for reverse. Of course, negative values aren't allowed here, so ensure that the last value is never less than 0. Also note that when a negative stepping is supplied, it is automatically inferred for the end index value to be 0 when not specified.

```
let array = [{0...30}]  
echo @array[30,-3..]
```

```
30 27 24 21 18 15 12 9 6 3 0
```

Process Expansions Also Support Slicing

Variables aren't the only elements that support slicing. Process expansions also support slicing.

```
echo $(cat file)[..10]  
echo @(cat file)[..10]
```

Flow Control

As Ion features an imperative paradigm, the order that statements are evaluated and executed is determined by various control flow keywords, such as `if`, `while`, `for`, `break`, and `continue`. Ion's control flow logic is very similar to POSIX shells, but there are a few major differences, such as that all blocks are ended with the `end` keyword; and the `do`/`then` keywords aren't necessary.

Conditionals

Conditionals in a language are a means of describing blocks of code that may potentially execute, so long as certain conditions are met. In Ion, as with every other language, we support this via **if statements**, but unlike POSIX shells, we have a cleaner syntax that will require less boilerplate, and increase readability.

If Statements

The **if** keyword in Ion is designated as a control flow keyword, which works similar to a builtin command. The **if** builtin will have it's supplied expression parsed and executed. The return status of the executed command will then be utilized as a boolean value. Due to the nature of how shells operate though, a logical **true** result is a **0** exit status, which is an exit status that commands return when no errors are reported. If the value is not zero, it is considered **false**. Sadly, we can't go back in time to tell early UNIX application developers that **1** should indicate success, and **0** should indicate a general error, so this behavior found in POSIX shells will be preserved.

We supply a number of builtin commands that are utilized for the purpose of evaluating expressions and values that we create within our shell. One of these commands is the **test builtin**, which is commonly found in other POSIX shells, and whose flags and operation should be identical. We also supply a **not** builtin, which may be convenient to use in conjunction with other commands in order to flip the exit status; and a **matches** builtin that performs a regex-based boolean match.

```
if test "foo" = $foo
  echo "Found foo"
else if matches $foo '[A-Ma-m]\w+'
  echo "we found a word that starts with A-M"
  if not matches $foo '[A]'
    echo "The word doesn't start with A"
  else
    echo "The word starts with 'A'"
  end
else
  echo "Incompatible word found"
end
```

A major distinction with POSIX shells is that Ion does not require that the if statement is followed with a **then** keyword. The **else if** statements are also written as two separate words, rather than as **elif** which is POSIX. And all blocks in Ion are ended with the **end**

keyword, rather than `fi` to end an if statement. There is absolutely zero logical reason for a shell language to have multiple different keywords to end different expressions.

Complete List of Conditional Builtins

- ☒ `and`
- ☒ `contains`
- ☒ `exists`
- ☒ `eq`
- ☐ `intersects`
- ☒ `is`
- ☒ `isatty`
- ☒ `matches`
- ☒ `not`
- ☒ `or`
- ☒ `test`
- ☐ `<` (Polish Notation)
- ☐ `<=` (Polish Notation)
- ☐ `>` (Polish Notation)
- ☐ `>=` (Polish Notation)
- ☐ `=` (Polish Notation)

Using the `&&` and `||` Operators

We also support performing conditional execution that can be performed within job execution, using the same familiar POSIX syntax. The `&&` operator denotes that the following command should only be executed if the previous command had a successful return status. The `||` operator is therefore the exact opposite. These can be chained together so that jobs can be skipped over and conditionally-executed based on return status. This enables succinctly expressing some patterns better than could be done with an if statement.

```
if test $foo = "foo" && test $bar = "bar"
  echo "foobar was found"
else
  echo "either foo or bar was not found"
end
```

```
test $foo = "foo" && test $bar = "bar" &&  
  echo "foobar was found" ||  
  echo "either foo or bar was not found"
```


Loops

Loops enable repeated execution of statements until certain conditions are met. There are currently two forms of loop statements: for loops, and while loops.

For Loops

For loops take an array of elements as the input; looping through each statement in the block with each element in the array. If the input is a string, however, that string will automatically coerce into a newline-delimited array.

```
for element in @array
  echo $element
end
```

Splitting Arguments

When working with strings that you would like to splice into multiple elements for iteration, see the splicing method, `@split`:

```
let value = "one two three four"
for element in @split(value)
  echo $element
end
```

By default, this will split a string by whitespace. Custom patterns may also be provided:

```
let value = "one,two,three,four"
for element in @split(value ',',')
  echo $element
end
```

A convenience method is also provided for `@split(value '\n')`: `@lines`

```
let file = $(cat file)
for line in @lines(file)
  echo = $line =
end
```

Breaking From Loops

Sometimes you may need to exit from the loop before the looping is finished. This is achievable using the `break` keyword.

```
for element in {1..=10}
  echo $element
  if test $element -eq 5
    break
  end
end
```

```
1
2
3
4
```

Continuing Loops

In other times, if you need to abort further execution of the current loop and skip to the next loop, the `continue` keyword serves that purpose.

```
for elem in {1..=10}
  if test $((elem % 2)) -eq 1
    continue
  end
  echo $elem
end
```

```
2
4
6
8
10
```

While Loops

While loops are useful when you need to repeat a block of statements endlessly until certain conditions are met. It works similarly to if statements, as it also executes a command and compares the exit status before executing each loop.

```
let value = 0
while test $value -lt 6
  echo $value
  let value += 1
end
```

```
0
1
2
3
4
5
```

Chunked Iterations

Chunked iterations allow fetching multiple values at a time.

```
for foo bar bazz in {1..=10}
  echo $foo $bar $bazz
end
```

```
1 2 3
4 5 6
7 8 9
10
```

Matches

Matches will evaluate each case branch, and execute the first branch which succeeds. A case which is `_` will execute if all other cases have failed.

```
match $string
  case "this"
    echo "do that"
  case "that"
    echo "else this"
  case _; echo "not found"
end
```

Matching string input with array cases

If the input is a string, and a case is an array, then a match will succeed if at least one item in the array is a match.

```
match five
  case [ one two three ]; echo "one of these matched"
  case [ four five six ]; echo "or one of these matched"
  case _; echo "no match found"
end
```

Matching array input with string cases

The opposite is true when the input is an array, and a case is a string.

```
match [ five foo bar ]
  case "one"; echo "this"
  case "two"; echo "that"
  case "five"; echo "found five"
  case _; echo "no match found"
end
```

Match guards

Match guards can be added to a match to employ an additional test

```
let foo = bar
match $string
  case "this" if eq $foo bar
    echo "this and foo = bar"
  case "this"
    echo "this and foo != bar"
  case _; echo "no match found"
end
```

Pipelines

Redirection

Redirection will write the output of a command to a file.

Redirect Stdout

```
command > stdout
```

Redirect Stderr

```
command ^> stderr
```

Redirect Both

```
command &> combined
```

Multiple Redirection

```
command > stdout ^> stderr &> combined
```

Concatenating Redirect

Instead of truncating and writing a new file with `>`, the file can be appended to with `>>`.

```
command > stdout  
command >> stdout
```

Pipe

Pipe Stdout

```
command | command
```

Pipe Stderr

```
command ^| command
```

Pipe Both

```
command &| command
```

Combined

```
command | command > stdout
```

Detaching processes

Send to background

```
command &
```

Disown (detach from shell)

```
command &!
```

Functions

Functions help scripts to reduce the amount of code duplication and increase readability. Ion supports the creation of functions with a similar syntax to other languages.

The basic syntax of functions is as follows:

```
fn square
  let x = "5"
  echo $(( x * x ))
end

square
square
```

Every statement between the `fn` and the `end` keyword is part of the function. On every function call, those statements get executed. That script would output "25" two times.

If you want the square of something that isn't five, you can add arguments to the function.

```
fn square x
  echo $(( x * x ))
end

square 3
```

Type checking

Optionally, you can add type hints into the arguments to make ion check the types of the arguments:

```
fn square x:int
  echo $(( x * x ))
end

square 3
square a
```

You'd get as output of that script:

```
9
ion: function argument has invalid type: expected int, found value 'a'
```


You can use any of the [supported types](#).

Multiple arguments

As another example:

```
fn hello name age:int hobbies:[str]
  echo "$name ($age) has the following hobbies:"
  for hobby in @hobbies
    echo "  $hobby"
  end
end

hello John 25 [ coding eating sleeping ]
```

Function piping

As with any other statement, you can pipe functions using `read`.

```
fn format_with pat
  read input
  echo $join(@split($input) $pat)
end

echo one two three four five | format_with "-"
```

Docstrings

Functions can be given a description with the following syntax:

```
fn square x -- Squares a single number
  echo $(( x * x ))
end
```

This description is then printed when `fn` is run without arguments.

Library usage:

When using Ion as a shell library, it is possible you may want to change the builtin functions associated with a Shell.

If you do this, all function calls will use the new builtins to run. meaning that if you removed the builtin function it the shell will try to find the command, and if you added a builtin, that will override any other command.

Advanced usage (THIS MAY BREAK ANY TIME)

```
fn square x:int; echo $(( x * x )); end  
square 22
```

```
484
```

Script Executions

Scripts can be created by designating Ion as the interpreter in the shebang line.

```
#!/usr/bin/env ion
```

Then writing the script as you would write it in the prompt. When finished, you can execute the shell by providing the path of the script to Ion as the argument, along with any additional arguments that the script may want to parse. Arguments can be accessed from the **@args** array, where the first element in the array is the name of the script being executed.

```
#!/usr/bin/env ion

if test $len(@args) -eq 1
    echo "Script didn't receive enough arguments"
    exit
end

echo Arguments: @args[1..]
```

Executing commands at start of interactive ion session

Commands can be executed at the start of an interactive ion session. This can be useful to set up environmental variables specific to ion, set aliases, set vim keybindings, etc.

Ion reads those initial commands from a file called "initrc". Ion looks for this file in its configuration folder according to the [xdg standard](#).

This typically results in the configuration path ~/.config/ion by default. So ion searches for the file initrc in the folder ~/.config/ion normally.

For example the following content of the initrc file causes ion to start with vim bindings enabled.

```
keybindings vi
```

You can change the location where ion looks for its configuration folder by setting the environmental variable **\$XDG_CONFIG_HOME** to a different path.

```
export XDG_CONFIG_HOME='/home/some_user/myconfig'
```

In this example ion will look at the path `/home/some_user/myconfig/ion` for the `initrc` file.

Signal Handling

- **SIGINT** (Ctrl + C): Interrupt the running program with a signal to terminate.
- **SIGTSTP** (Ctrl + Z): Send the running job to the background, pausing it.

Job Control

Disowning Processes

Ion features a `disown` command which supports the following flags:

- **-r**: Remove all running jobs from the background process list.
- **-h**: Specifies that each job supplied will not receive the `SIGHUP` signal when the shell receives a `SIGHUP`.
- **-a**: If no job IDs were supplied, remove all jobs from the background process list.

Unlike Bash, job arguments are their specified job IDs.

Foreground & Background Tasks

When a foreground task is stopped with the **Ctrl+Z** signal, that process will be added to the background process list as a stopped job. When a supplied command ends with the **&** operator, this will specify to run the task the background as a running job. To resume a stopped job, executing the `bg <job_id>` command will send a `SIGCONT` to the specified job ID, hence resuming the job. The `fg` command will similarly do the same, but also set that task as the foreground process. If no argument is given to either `bg` or `fg`, then the previous job will be used as the input.

Exiting the Shell

The `exit` command will exit the shell, sending a `SIGTERM` to any background tasks that are still active. If no value is supplied to `exit`, then the last status that the shell received will be used as the exit status. Otherwise, if a numeric value is given to the command, then that value will be used as the exit status.

Suspending the Shell

While the shell ignores `SIGTSTP` signals, you can forcefully suspend the shell by executing the `suspend` command, which forcefully stops the shell via a `SIGSTOP` signal.

Builtin commands

bg - sends jobs to background

SYNOPSIS

```
bg PID
```

DESCRIPTION

bg sends the job to the background resuming it if it has stopped.

bool - Returns true if the value given to it is equal to '1' or 'true'.

SYNOPSIS

```
bool VALUE
```

DESCRIPTION

Returns true if the value given to it is equal to '1' or 'true'.

cd - Change directory.

SYNOPSIS

```
cd DIRECTORY
```

DESCRIPTION

Without arguments cd changes the working directory to your home directory.

With arguments cd changes the working directory to the directory you provided.

contains - check if a given string contains another one

SYNOPSIS

```
contains <PATTERN> tests...
```

DESCRIPTION

Returns 0 if the first argument contains any of the other ones, else returns 1

dir-depth - set the dir stack depth

SYNOPSIS

```
dir-depth [DEPTH]
```

DESCRIPTION

If DEPTH is given, set the dir stack max depth to DEPTH, else remove the limit

dirs - prints the directory stack

SYNOPSIS

```
dirs
```

DESCRIPTION

dirs prints the current directory stack.

disown - disown processes

SYNOPSIS

```
disown [ --help | -r | -h | -a ][PID...]
```

DESCRIPTION

Disowning a process removes that process from the shell's background process table.

OPTIONS

- r Remove all running jobs from the background process list.
- h Specifies that each job supplied will not receive the SIGHUP signal when the shell receives a SIGHUP.
- a If no job IDs were supplied, remove all jobs from the background process list.

drop - delete some variables or arrays

SYNOPSIS

```
drop VARIABLES...
```

DESCRIPTION

Deletes the variables given to it as arguments. The variables name must be supplied.

Instead of '\$x' use 'x'.

echo - display text

SYNOPSIS

```
echo [ -h | --help ] [-e] [-n] [-s] [STRING]...
```

DESCRIPTION

Print the STRING(s) to standard output.

OPTIONS

-e
enable the interpretation of backslash escapes

-n
do not output the trailing newline

-s
do not separate arguments with spaces

Escape Sequences

When the **-e** argument is used, the following sequences will be interpreted:

**** backslash
\a alert (BEL)
\b backspace (BS)
\c produce no further output
\e escape (ESC)
\f form feed (FF)
\n new line
\r carriage return
\t horizontal tab (HT)
\v vertical tab (VT)

ends-with - check if a given string ends with another one

SYNOPSIS

```
ends-with <PATTERN> tests...
```

DESCRIPTION

Returns 0 if the first argument ends with any of the other ones, else returns 1

eval - evaluates the specified commands

SYNOPSIS

```
eval COMMANDS...
```

DESCRIPTION

eval evaluates the given arguments as a command. If more than one argument is given,
all arguments are joined using a space as a separator.

exec - replace the shell with the given command

SYNOPSIS

```
exec [-ch] [--help] [command [arguments ...]]
```

DESCRIPTION

Execute <command>, replacing the shell with the specified program.
The <arguments> following the command become the arguments to
<command>.

OPTIONS

-c Execute command with an empty environment.

exists - check whether items exist

SYNOPSIS

```
exists [EXPRESSION]
```

DESCRIPTION

Checks whether the given item exists and returns an exit status of 0 if it does, else 1.

OPTIONS

```
-a ARRAY
    array var is not empty

-b BINARY
    binary is in PATH

-d PATH
    path is a directory
    This is the same as test -d

-f PATH
    path is a file
    This is the same as test -f

--fn FUNCTION
    function is defined

-s STRING
    string var is not empty

STRING
    string is not empty
    This is the same as test -n
```

EXAMPLES

Test if the file exists:

```
exists -f FILE && echo 'The FILE exists' || echo 'The FILE does not exist'
```

Test if some-command exists in the path and is executable:

```
exists -b some-command && echo 'some-command exists' || echo 'some-command does not exist'
```

Test if variable exists AND is not empty

```
exists -s myVar && echo "myVar exists: $myVar" || echo 'myVar does not exist or is empty'
```

NOTE: Don't use the '\$' sigil, but only the name of the variable to check

Test if array exists and is not empty

```
exists -a myArr && echo "myArr exists: @myArr" || echo 'myArr does not exist or is empty'
```

NOTE: Don't use the '@' sigil, but only the name of the array to check

Test if a function named 'myFunc' exists

```
exists --fn myFunc && myFunc || echo 'No function with name myFunc found'
```

AUTHOR

Written by Fabian Würfl.

Heavily based on implementation of the test builtin, which was written by Michael Murphy.

exit - exit the shell

SYNOPSIS

```
exit
```

DESCRIPTION

Makes ion exit. The exit status will be that of the last command executed.

false - does nothing unsuccessfully

SYNOPSIS

```
false
```

DESCRIPTION

Sets the exit status to 1.

fg - bring job to the foreground

SYNOPSIS

```
fg PID
```

DESCRIPTION

fg brings the specified job to foreground resuming it if it has stopped.

fn - print a short description of every defined function

SYNOPSIS

```
fn [ -h | --help ]
```

DESCRIPTION

Prints all the defined functions along with their help, if provided

help - get help for builtins

SYNOPSIS

```
help [BUILTIN]
```

DESCRIPTION

Get the short description for BUILTIN. If no argument is provided, list all the builtins

history - print command history

SYNOPSIS

```
history [option]
```

DESCRIPTION

Prints or manipulate the command history.

OPTIONS:

+inc_append: Append each command to history as entered.

-inc_append: Default, do not append each command to history as entered.

+shared: Share history between shells using the same history file, implies inc_append.

-shared: Default, do not share shell history.

+duplicates: Default, allow duplicates in history.

-duplicates: Do not allow duplicates in history.

eq, is - checks if two arguments are the same

SYNOPSIS

```
is [ -h | --help ] [not]
```

DESCRIPTION

Returns 0 if the two arguments are equal, and 1 otherwise.

OPTIONS

not

returns 0 if the two arguments are not equal.

isatty - checks if the provided file descriptor is a tty

SYNOPSIS

```
isatty [FD]
```

DESCRIPTION

Returns 0 if the supplied file descriptor is a tty, and 1 otherwise.

jobs - list all jobs running in the background

SYNOPSIS

```
jobs
```

DESCRIPTION

Prints a list of all jobs running in the background.

matches - checks if the second argument contains any proportion of the first

SYNOPSIS

```
matches VALUE VALUE
```

DESCRIPTION

Makes the exit status equal 0 if the first argument contains the second. Otherwise matches makes the exit status equal 1.

EXAMPLES

Returns true:

```
matches xs x
```

Returns false:

```
matches x xs
```


math - Floating-point calculator

SYNOPSIS

```
math [EXPRESSION]
```

DESCRIPTION

Evaluates arithmetic expressions

SPECIAL EXPRESSIONS

help (only in interactive mode)
prints this help text

--help (only in non-interactive mode)
prints this help text

exit (only in interactive mode)
exits the program

NOTATIONS

infix notation
e.g. 3 * 4 + 5

polish notation
e.g. + * 3 4 5

EXAMPLES

Add two plus two in infix notation
math 2+2

Add two plus two in polish notation
math + 2 2

AUTHOR

Written by Hunter Goldstein.

popd - shift through the directory stack

SYNOPSIS

```
popd
```

DESCRIPTION

popd removes the top directory from the directory stack and changes the working directory to the new top directory.
pushd adds directories to the stack.

pushd - push a directory to the directory stack

SYNOPSIS

```
pushd DIRECTORY
```

DESCRIPTION

pushd pushes a directory to the directory stack.

random - generate a random number

SYNOPSIS

```
random  
random START END
```

DESCRIPTION

random generates a pseudo-random integer. IT IS NOT SECURE.
The range is half open and depends on what arguments you pass.
If no arguments are given the range is [0, 32767).
If two arguments are given the range is [START, END).

read - read a line of input into some variables

SYNOPSIS

```
read VARIABLES...
```

DESCRIPTION

For each variable reads from standard input and stores the results in the variable.

set - Set or unset values of shell options and positional parameters.

SYNOPSIS

```
set [ --help ] [-e | +e] [-x | +x] [-o [vi | emacs]] [- | --] [STRING]...
```

DESCRIPTION

Shell options may be set using the '-' character, and unset using the '+' character.

OPTIONS

- e Exit immediately if a command exits with a non-zero status.
- o Specifies that an argument will follow that sets the key map. The keymap argument may be either `vi` or `emacs`.
- x Specifies that commands will be printed as they are executed.
- Following arguments will be set as positional arguments in the shell. If no argument are supplied, arguments will be unset.
- Following arguments will be set as positional arguments in the shell. If no arguments are supplied, arguments will not be unset.

source - evaluates given file

SYNOPSIS

```
source FILEPATH
```

DESCRIPTION

Evaluates the commands in a specified file in the current shell. All changes in shell variables will affect the current shell because of this.

starts-with - check if a given string starts with another one

SYNOPSIS

```
starts-with <PATTERN> tests...
```

DESCRIPTION

Returns 0 if the first argument starts with any of the other ones, else returns 1

status - Evaluates the current runtime status

SYNOPSIS

```
status [ -h | --help ] [-l] [-i]
```

DESCRIPTION

With no arguments status displays the current login information of the shell.

OPTIONS

-l

returns true if the shell is a login shell. Also --is-login.

-i

returns true if the shell is interactive. Also --is-interactive.

-f

prints the filename of the currently running script or else stdio. Also --current-filename.

suspend - suspend the current shell

SYNOPSIS

```
suspend
```

DESCRIPTION

Suspends the current shell by sending it the SIGTSTP signal, returning to the parent process. It can be resumed by sending it SIGCONT.

test - perform tests on files and text

SYNOPSIS

```
test [EXPRESSION]
```

DESCRIPTION

Tests the expressions given and returns an exit status of 0 if true, else 1.

OPTIONS

--help

prints this help text

-n STRING

the length of STRING is nonzero

STRING

equivalent to -n STRING

-z STRING

the length of STRING is zero

STRING = STRING

the strings are equivalent

STRING != STRING

the strings are not equal

INTEGER -eq INTEGER

the integers are equal

INTEGER -ge INTEGER

the first INTEGER is greater than or equal to the second INTEGER

INTEGER -gt INTEGER

the first INTEGER is greater than the second INTEGER

INTEGER -le INTEGER

the first INTEGER is less than or equal to the second INTEGER

INTEGER -lt INTEGER

the first INTEGER is less than the second INTEGER

INTEGER -ne INTEGER

the first INTEGER is not equal to the second INTEGER

FILE -ef FILE

both files have the same device and inode numbers

FILE -nt FILE

the first FILE is newer than the second FILE

FILE -ot FILE

the first file is older than the second FILE

```
-b FILE
    FILE exists and is a block device

-c FILE
    FILE exists and is a character device

-d FILE
    FILE exists and is a directory

-e FILE
    FILE exists

-f FILE
    FILE exists and is a regular file

-h FILE
    FILE exists and is a symbolic link (same as -L)

-L FILE
    FILE exists and is a symbolic link (same as -h)

-r FILE
    FILE exists and read permission is granted

-s FILE
    FILE exists and has a file size greater than zero

-S FILE
    FILE exists and is a socket

-w FILE
    FILE exists and write permission is granted

-x FILE
    FILE exists and execute (or search) permission is granted
```

EXAMPLES

Test if the file exists:

```
test -e FILE && echo "The FILE exists" || echo "The FILE does not exist"
```

Test if the file exists and is a regular file, and if so, write to it:

```
test -f FILE && echo "Hello, FILE" >> FILE || echo "Cannot write to a
directory"
```

Test if 10 is greater than 5:

```
test 10 -gt 5 && echo "10 is greater than 5" || echo "10 is not greater
than 5"
```

Test if the user is running a 64-bit OS (POSIX environment only):

```
test $(getconf LONG_BIT) = 64 && echo "64-bit OS" || echo "32-bit OS"
```

AUTHOR

Written by Michael Murphy.

true - does nothing sucessfully

SYNOPSIS

true

DESCRIPTION

Sets the exit status to 0.

wait - wait for a background job

SYNOPSIS

wait

DESCRIPTION

Wait for the background jobs to finish

which, type - locate a program file in the current user's path

SYNOPSIS

which PROGRAM

DESCRIPTION

The which utility takes a list of command names and searches for the alias/builtin/function/executable that would be executed if you ran that command.

Command history

The `history` builtin command can be used to display the command history:

- to display the entire command history, type `history` ;
- if you're only interested in the last N entries, type `history | tail -N`.

Its behavior can be changed via various local variables (see [Variables](#) below).

Ion's history file is located by default in `$HOME/.local/share/ion/history`.

Unlike other shells, Ion by default saves repeated commands only once:

```
# echo "Hello, world!"
Hello, world!
# true
# true
# false
# history
echo "Hello, world!"
true
false
```

The REPL provides the following useful shortcuts for history searching:

- Ctrl + s => forward search history ;
- Ctrl + r => reverse search history ;
- Ctrl + f => accept autosuggestion ;
- Ctrl + u => delete content ;
- Ctrl + c => interrupt command .

Variables

The following local variables can be used to modify Ion's history behavior:

HISTFILE

The file into which the history should be saved. At Ion' startup, the history will be read from this file, and when it exits, the session's history will be appended to this file.

Default value: `$HOME/.local/share/ion/history`

HISTFILE_ENABLED

Whether the history should be read from/written into the file specified by `HISTFILE`.

Default value: `1`

A value of `1` means yes, everything else means no.

HISTFILE_SIZE

The maximum number of lines kept in the history file when flushed from memory.

Default value: `100000`

It is useless to set this to a higher value than `HISTORY_SIZE`. Ideally, those variables would have the same value, since this would otherwise result in loss of information on history write to disk, which might not be worth it given the nowadays cheap hardware space.

(Currently ignored)

HISTORY_IGNORE

Which commands should *not* be saved in the history.

Default value: `[ no_such_command whitespace duplicates ]`

Each element of the array can take one of the following options:

- `all`
All commands are ignored, nothing will be saved in the history.
- `no_such_command`
Commands which return `NO_SUCH_COMMAND` will not be saved in the history.
- `whitespace`
Commands which start with a [whitespace character](#) will not be saved in the history.
- `regex:xxx`
Where xxx is treated as a [regular expression](#). Commands which match this regular expression will not be saved in the history.
- `duplicates`
All preceding duplicate commands are removed/ignored from the history after a matching command is entered.

Specifying an empty array, means that all commands will be saved.

Notes

- You can specify as many elements as you want.
- Any invalid elements will be silently ignored. They will still be present in the array though.
- You can also specify as many regular expressions as you want (each as a separate element).
- However, note that any command that matches **at least one** element will be ignored.
- (Currently,) there is no way to specify commands which should always be saved.
- When specifying **regex**-elements, it is suggested to surround them with single-quotes (')
- As all variables, `HISTORY_IGNORE` is not saved between sessions. It is suggested to set it via ions init file.
- The `let HISTORY_IGNORE = [ .. ]` command itself is **not effected** except if the assignment command starts with a whitespace and the **whitespace** element is specified in this assignment.

See the following example:

```
# echo @HISTORY_IGNORE

# let HISTORY_IGNORE = [ all ] # saved
# let HISTORY_IGNORE = [ whitespace ] # saved
# true # ignored
# let HISTORY_IGNORE = [ ] # saved
# let HISTORY_IGNORE = [ whitespace ] # ignored
# history
echo @HISTORY_IGNORE
let HISTORY_IGNORE = [ all ] # saved
let HISTORY_IGNORE = [ whitespace ] # saved
let HISTORY_IGNORE = [ ] # saved
```

Examples

```
# let HISTORY_IGNORE = [ no_such_command ]
# true # saved
# true # saved
# false # saved
# trulse # ignored
```

```
# let HISTORY_IGNORE = [ 'regex:.*' ] # behaves like 'all'
# true # ignored
# true # ignored
# false # ignored
# true # ignored
```

Tip

I like to add `regex:#ignore$` to my `HISTORY_IGNORE`. That way, whenever I want to ignore a command on the fly, I just need to add `#ignore` to the end of the line.

HISTORY_SIZE

The maximum number of lines contained in the command history in-memory.

Default value: `1000`

Ideally, this value should be the same as `HISTFILE_SIZE` (see `HISTFILE_SIZE` for details).

(Currently ignored)

HISTORY_TIMESTAMP

Whether a corresponding timestamp should be recorded along with each command.

The timestamp is indicated with a `#` and is unformatted as the seconds since the unix epoch.

Default value: `0`

Possible values are `0` (disabled) and `1` (enabled).